# Lecture #10: From Coloring to Eulerian Structure

Finalizing vertex coloring: planarity and the Four Colour Theorem

Edge coloring and the chromatic index $\chi'(G)$

Maximum degree, Vizing's Theorem, and Class 1/Class 2 graphs

Bipartite edge coloring: Kőnig's Line Coloring Theorem

Eulerian graphs beyond the basic definitions

Constructing Euler tours efficiently: Hierholzer's Algorithm

# Closing Vertex Coloring: What Planarity Gives Us

Planarity restricts how densely vertices can interact — this also restricts coloring.

Every planar graph can be properly vertex-colored using at most four colors.

The bound is universal: it does not say that every planar graph needs four colors.

Examples: trees need ≤2; odd cycles need 3; some planar graphs genuinely require 4.

**Planar G ⇒ χ(G) ≤ 4**

**Planar + triangle-free ⇒ χ(G) ≤ 3**

**Planar + bipartite ⇒ χ(G) ≤ 2**

*Interpretation: forbidding triangles strengthens the planar coloring bound from four colors to three.*

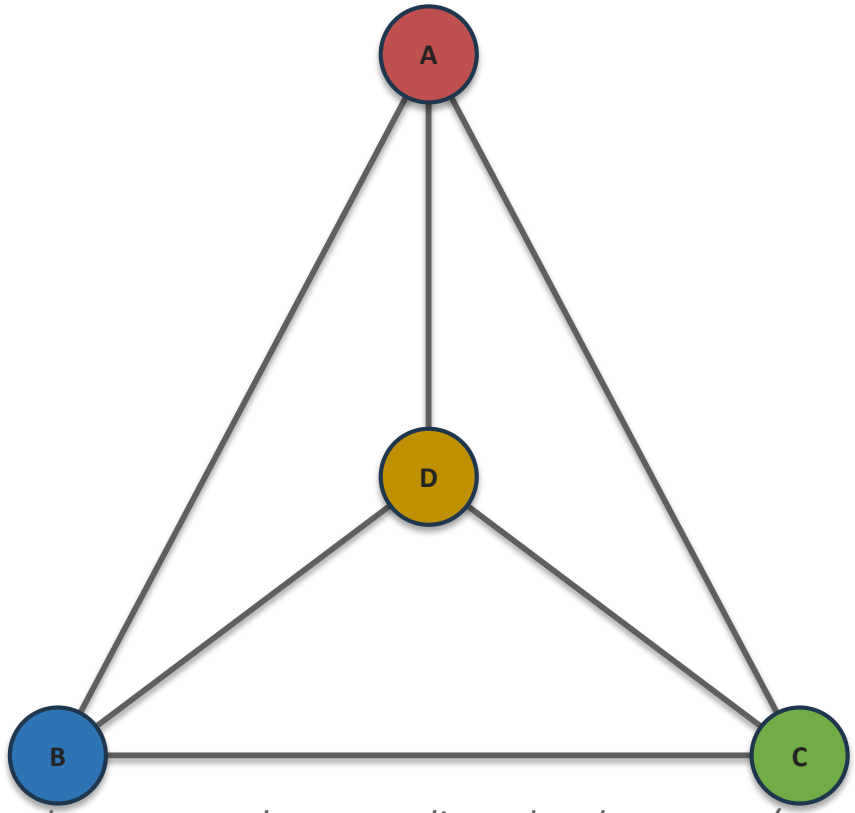
# Four Colour Theorem — The Important Point

Four Colour Theorem: every planar graph admits a proper vertex coloring with at most four colors.

Equivalent map interpretation: four colors suffice to color regions of any planar map so that regions sharing a boundary segment receive different colors.

The theorem is an upper bound, not a formula for $\chi(G)$.

It was historically difficult: the first accepted proof relied heavily on computer verification.




*In words: no matter how complicated a planar map/graph is, four vertex colors are always sufficient.*

**$K_4$ is planar and $\chi(K_4)=4$**

# Example: Use Planarity Before You Color

Suppose G is a connected planar graph with 10 vertices and 15 edges, and G contains no triangles.

Question: what useful upper bound can we place on $\chi(G)$ without constructing a coloring?

Because G is triangle-free and planar, the stronger planar result gives $\chi(G) \leq 3$.

If G is also known to contain an odd cycle, then $\chi(G) \geq 3$.

Therefore $\chi(G)=3$.

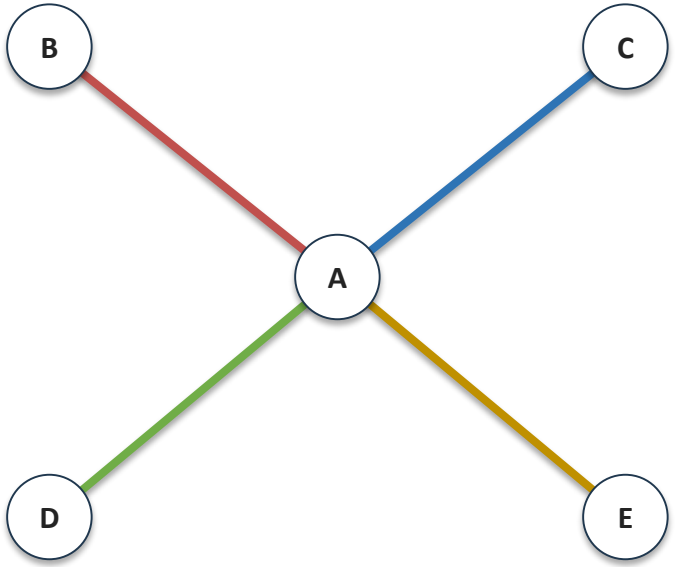
**triangle-free planar $\Rightarrow$ $\chi(G) \leq 3$**

**odd cycle present $\Rightarrow$ $\chi(G) \geq 3$**

**$\therefore$ $\chi(G) = 3$**

# Why Edge Coloring? When the Conflict Lives on the Edges

Vertex coloring: conflicting entities are vertices.
Edge coloring: conflicting tasks/relationships are edges.




### Match Scheduling

Vertices = teams

Edge uv = a match between teams u and v

Color = time slot

Two matches sharing a team cannot occur in the same slot.

**If the busiest team has Δ matches, are Δ time slots always enough?**

$$\chi'(G) \geq \Delta(G)$$

# Edge Coloring: Formalizing the Scheduling Conflict

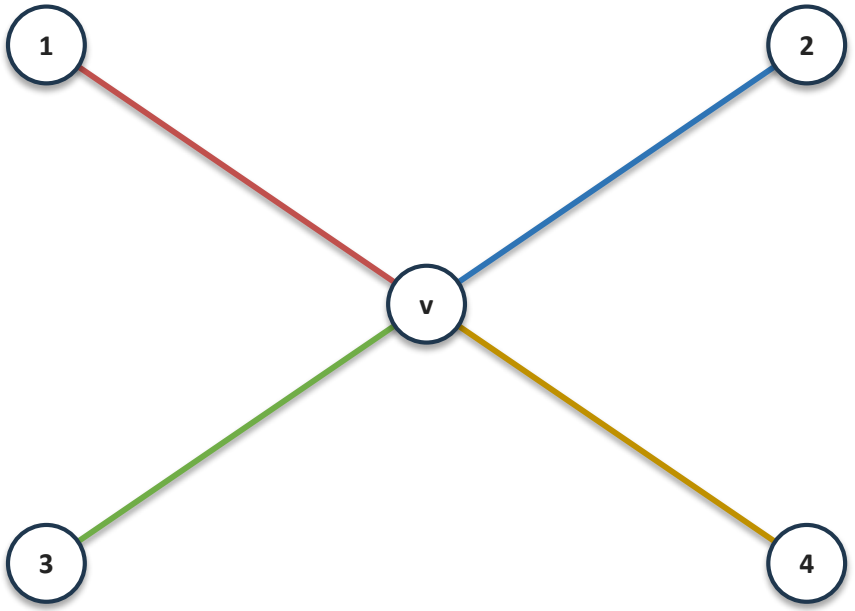
A proper edge coloring assigns colors to edges so that any two edges sharing an endpoint receive different colors.

The minimum number of colors required is the chromatic index $\chi'(G)$.

At a vertex of degree d, its d incident edges must all have different colors.

Hence maximum degree immediately gives a lower bound.

$$\chi'(G) \geq \Delta(G)$$



# Vizing's Theorem: Only Two Possibilities for Simple Graphs

$$\Delta(G) \leq \chi'(G) \leq \Delta(G)+1$$

***A simple graph never needs more than one extra edge color beyond its maximum degree.***

Class 1 graph: $\chi'(G)=\Delta(G)$

Class 2 graph: $\chi'(G)=\Delta(G)+1$

The theorem is powerful because a potentially difficult optimization is squeezed into only two possibilities.

But deciding which of the two values applies can still require structural reasoning.

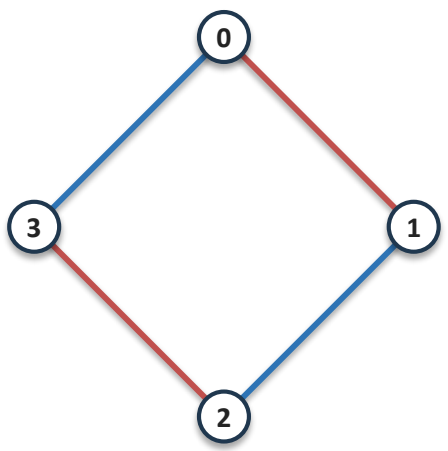
# Worked Example: Same Δ, Different Chromatic Index

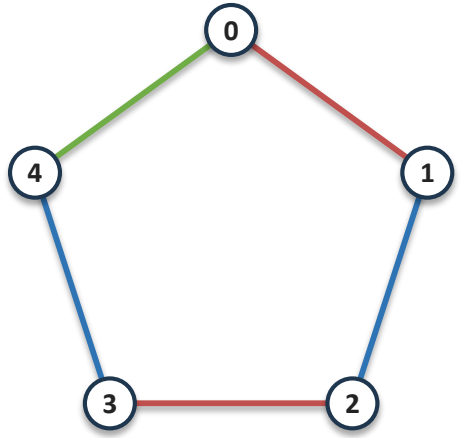
Compare $C_4$ and $C_5$. Both have maximum degree $\Delta=2$.

$C_4$: alternate two edge colors around the cycle ⇒ $\chi'(C_4)=2$. It is Class 1.

$C_5$: alternating two colors fails at the closing edge; a third color is necessary ⇒ $\chi'(C_5)=3$. It is Class 2.

General pattern: even cycles have $\chi'=2$; odd cycles have $\chi'=3$.




**$C_4$: $\chi'=2=\Delta$**

**$C_5$: $\chi'=3=\Delta+1$**

# Bipartite Edge Coloring: A Stronger Exact Result

Kőnig's Line Coloring Theorem:

For every bipartite graph G, the chromatic index equals its maximum degree.

So a bipartite graph is always Class 1.

This is stronger than merely knowing the Vizing interval {Δ, Δ+1}.

**G bipartite ⇒ χ′(G)=Δ(G)**

*For bipartite graphs, the lower bound Δ is always achievable—no extra color is ever needed.*

If edges represent jobs requiring both of their endpoint resources, Δ time slots always suffice when the resource graph is bipartite.

# Worked Example: Edge-Color $K_{3,3}$ with Exactly Three Colors

$K_{3,3}$ is bipartite and $\Delta=3$, so $\chi'=3$ immediately.

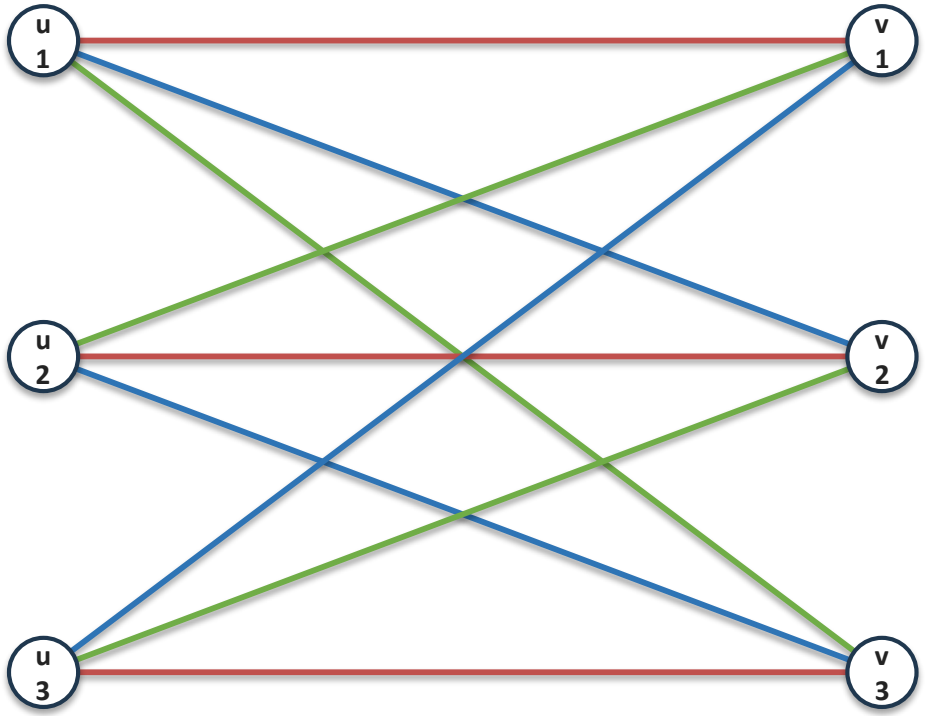
One construction decomposes the 9 edges into three perfect matchings:

Color 1: $(u_1,v_1)$, $(u_2,v_2)$, $(u_3,v_3)$

Color 2: $(u_1,v_2)$, $(u_2,v_3)$, $(u_3,v_1)$

Color 3: $(u_1,v_3)$, $(u_2,v_1)$, $(u_3,v_2)$

Each color class is a matching, so no same-colored edges meet.



# A Structural Connection: Edge Coloring as Vertex Coloring

The line graph L(G) converts every edge of G into a vertex.

Two vertices in L(G) are adjacent exactly when the corresponding edges in G share an endpoint.

Therefore an edge coloring of G is precisely a vertex coloring of L(G).

$$\chi'(G) = \chi(L(G))$$

*In words: coloring edges of G is exactly the same problem as coloring vertices of its line graph L(G).*

This is useful conceptually: edge-conflict problems can be transformed into ordinary vertex-conflict problems.

# Moving On to Traversal: Eulerian Graphs

Euler trail: uses every edge exactly once.

Euler circuit: an Euler trail that starts and ends at the same vertex.

For a connected undirected graph:

- Euler circuit ⇔ every vertex has even degree.
- Euler trail but not a circuit ⇔ exactly two vertices have odd degree.

The key structural idea is parity, not the number of cycles.

**0 odd vertices → circuit**

**2 odd vertices → open trail**

**>2 odd vertices → impossible**

*The existence of an Euler traversal is completely determined by connectivity and the parity of vertex degrees.*

# Why Parity Controls an Euler Tour

At an intermediate vertex, every arrival along an unused edge must be paired with a departure along another unused edge.

Thus incident edges are consumed in pairs.

In a closed Euler circuit, this pairing occurs at every vertex → all degrees are even.

For an open Euler trail, the start has one unmatched departure and the end one unmatched arrival → exactly two odd-degree vertices.

*Key idea: except at the two possible endpoints of an open trail, edges at each visited vertex must be used in enter–leave pairs.*

**Handshaking Lemma ⇒ number of odd-degree vertices is always even**

# Example: Diagnose Before Constructing

Degrees:

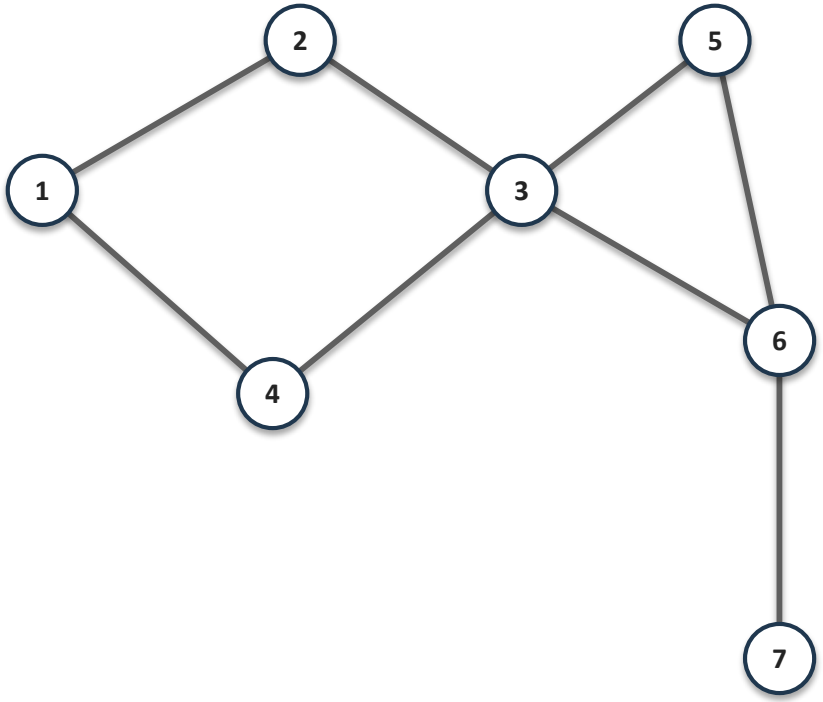
deg(1)=2, deg(2)=2, deg(3)=4, deg(4)=2

deg(5)=2, deg(6)=3, deg(7)=1

Exactly two vertices are odd: 6 and 7.

Therefore the graph has an Euler trail, but no Euler circuit.

Any Euler trail must start at one of {6,7} and end at the other.



# Not-So-Simple Twist: What Does One Added Edge Change?

Suppose a connected graph currently has four odd-degree vertices: {a,b,c,d}.

Adding one edge flips the parity of exactly its two endpoints.

If we add an edge between two odd vertices, both become even: the number of odd vertices drops from 4 to 2.

The resulting graph can then have an Euler trail (provided connectivity is retained, which it is after adding an edge).

A second suitable edge pairing the remaining odd vertices would make all degrees even and produce an Eulerian graph.

**Adding edge uv: parity(deg u) and parity(deg v) both flip**

**4 odd → add odd–odd edge → 2 odd**

**2 odd → add odd–odd edge → 0 odd**

# Constructing the Tour: Why Hierholzer Beats Greedy Edge Picking

A naïve choice of unused edges can trap us before all edges are covered.

Hierholzer's idea: in an Eulerian graph, follow unused edges until you return to the start — this forms a cycle.

If unused edges remain, choose a vertex on the current cycle incident to an unused edge, build another closed cycle, and splice it in.

Repeat until every edge is included.

**Time complexity: O(V + E)**

Implementation idea:

- adjacency list + used-edge flags
- stack for current traversal
- output circuit in reverse as vertices are exhausted

*Hierholzer builds closed cycles from unused edges and splices them together until every edge is used once.*

# Example: Hierholzer Step by Step

All vertices have even degree, so an Euler circuit exists.

Start at 1:

First closed cycle: 1 → 2 → 3 → 1

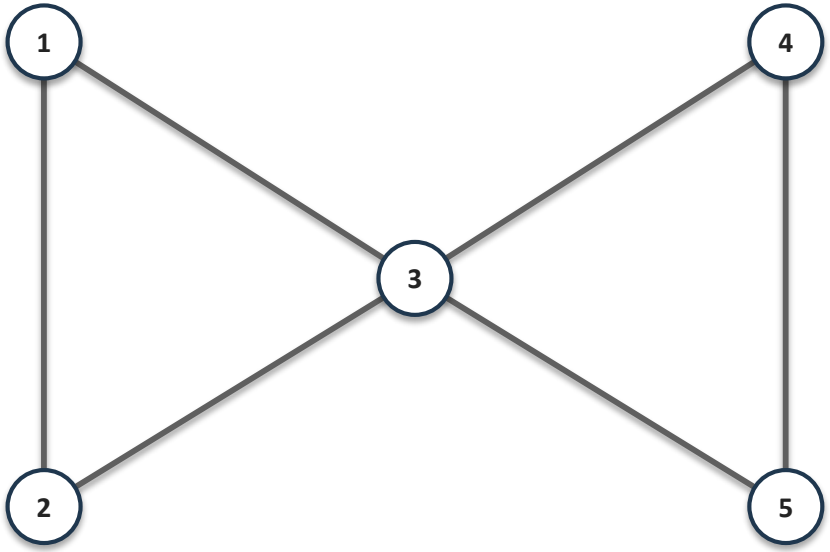
Vertex 3 on that cycle still has unused incident edges.

Second cycle from 3: 3 → 4 → 5 → 3

Splice at vertex 3:

1 → 2 → 3 → 4 → 5 → 3 → 1

Every edge appears exactly once.



# Lecture #10 — Takeaways

Planarity constrains vertex coloring: $\chi(G) \leq 4$ for every planar graph.

Edge coloring is governed tightly by maximum degree: $\Delta \leq \chi' \leq \Delta + 1$.

Bipartite graphs achieve the lower bound exactly: $\chi' = \Delta$.

Line graphs connect edge coloring back to vertex coloring.

Eulerian traversal is governed by degree parity.

Hierholzer gives a linear-time construction of an Euler tour.